

Multimodal Fashion & Context Retrieval

Glance ML Internship Assignment — Submission

1. Approaches

Possible ways to solve this problem, their tradeoffs, and when each is the right call:

Approach	How it works	Good for	Weakness
Keyword / filename matching	Match query words against filenames or manual tags	Trivial to build	Explicitly disallowed by the PRD; no generalization at all
Trained multi-label classifier + filter	Train a CNN on Fashionpedia's categories/attributes, filter images by predicted labels	Precise on labels seen during training	Zero-shot is exactly what it lacks — can't handle open vocabulary ("bright yellow raincoat") or scene/vibe language at all
Vanilla CLIP, single global embedding	Encode whole image + query into one shared vector space, rank by cosine similarity	Genuinely zero-shot; strong on scene/style/vibe language	The PRD's own hint: pooling the whole image into one vector loses compositional structure — "red tie, white shirt" and "white tie, red shirt" embed almost identically
Region-based detection + captioning	Run/train an object detector, caption each region, embed each caption separately	Best possible fine-grained compositional accuracy	Requires running or training a detector at index time; heavier pipeline, more failure surface, more engineering than the PRD asks for
Chosen: fashion-domain CLIP (whole-image + per-garment-region) + structured attributes derived from Fashionpedia's own segmentation, hybrid re-ranking	Detailed in Section 2	Genuinely zero-shot <i>and</i> compositional, using labels the dataset already provides — no detector training, no new annotation	Structured signal is capped by the category/color vocabulary and by corpus size; both measured explicitly below rather than assumed

2. Short Write-up on Chosen Approach

2.0 Dataset composition — does it actually cover the PRD's 3 required axes?

The PRD's dataset requirement is variation across **Environment** (office, urban street, park, home), **Clothing Types** (formal, casual, outerwear), and **Color Theory** (a wide palette). Measured directly against the 3,200-image indexed corpus, not assumed:

- **Environment:** all 4 named environments are present, but unevenly — runway (54.8%) and studio (22.6%) dominate; street (11.2%) and park (8.9%) are meaningful; **office (0.2%, 8 images) and home (0.3%, 9 images) are severely underrepresented**, since Fashionpedia is a runway/editorial fashion dataset, not a general street-style corpus. This is reported honestly because it directly explains PRD eval query #2's strong score: with only 8 true office-tagged images, that query mostly succeeds via the style="formal" signal (543 images, 17.0%), not abundant genuine office-scene matches.
- **Clothing Types:** all 4 named categories present and reasonably balanced (sporty 44.7%, casual 26.1%, formal 17.0%, outerwear 12.2%), across 46 distinct garment/part categories actually observed.
- **Color Theory:** 36 distinct colors extracted and observed (via the Lab-space classifier, Section 2.1) — from common (charcoal, black, gray, unsurprising for runway/editorial photography) to rare (olive and forest green: 7 instances each; yellow: 13, the 3rd-rarest color in the corpus). This is the direct explanation for Section 2.5's finding that no yellow coat exists in this corpus.

2.1 Architecture

Part A — The Indexer (`indexer/`)

1. `dataset.py` loads Fashionpedia's COCO-style annotations — segmentation polygons and category labels already exist per garment instance, so no new labeling was needed. It also exposes every image with *no* annotation, so the searchable corpus isn't capped at the labeled subset.
2. `embed.py` uses `patrickjohncyh/fashion-clip` (CLIP fine-tuned on ~800K fashion image/text pairs via HuggingFace `transformers`) — swapped in from generic `clip-vit-B-32` after verifying it discriminates fashion color/garment language measurably better. It embeds the **whole image** (captures scene/setting/style — "office", "park bench", "casual weekend") and **each garment crop separately**, padded 15% beyond its bounding box — this per-garment embedding is the core fix for compositionality (Section 2.2).
3. `color_extract.py` computes the color(s) of each garment instance from its segmentation mask (mask eroded 2px inward to avoid background/skin bleed, then median RGB), and `common/colors.py` names it by **nearest-neighbor search in CIE Lab space** against a 35-color fashion palette — not hand-tuned HSV thresholds. Lab space is perceptually uniform, so it resolves exactly the shade pairs fashion queries hinge on: burgundy vs. maroon, khaki vs. olive vs. sage, navy vs. denim. Regression-tested (22/22 cases, including a real failure caught during development — Section 2.5). `common/colors.py` also defines **chromatic families** (e.g. `{blue, navy, denim, sky blue}`) so a query for "a blue shirt" gets full credit for an exact "blue" label and graded partial credit for a family member like "denim". A **second color** is reported when a garment shows real internal color spread — measured at ~25% of garment-category instances in this corpus (a striped or color-blocked garment medians to one blended, often-wrong color otherwise).
4. `relations.py` derives one spatial/relational attribute — **layered** — from bounding-box overlap between same-body-region garment instances (e.g. a jacket overlapping a shirt). Scoped honestly: bounding boxes alone can't reliably give true z-order ("tucked in" vs. "worn over"), so this captures co-occurrence/layering, not full occlusion reasoning.
5. `scene_tag.py` is one generic zero-shot tagger, reused for three prompt banks: scene (office/street/park/home/studio/runway/beach/cafe/gym), style (formal/casual/outerwear/sporty), and weather (rainy/sunny/cold/indoor) — cosine-matched against the image embedding. This is the "locations and weather" extension implemented as **place type**, not city names — nothing in a street-style photo

honestly reveals which city it was taken in, so a city-name bank would fabricate structure CLIP can't actually ground.

6. **build_index.py** runs two passes: annotated images get full structured metadata (categories, colors, relations, scene/style/weather); the remaining unannotated images on disk get whole-image embedding + scene/style/weather tags only — growing the searchable corpus from 1,158 to **3,200 images**. Both live in **Chroma** — two persistent collections, one `pip install`, metadata filtering built in — per the PRD's explicit instruction to spend ML effort, not engineering effort, on the vector store.

Part B — The Retriever (retriever/)

1. **query_parser.py** scans the query for known garment/color words (vocabulary cached from the annotation file, never hand-maintained), scene/style/weather keywords, and layering phrases ("layered", "over", "under", "tucked"). Garment words the hardcoded synonym dictionary misses ("windbreaker", "parka", "loafers", "gown") are resolved **zero-shot** by `vocab_resolver.py` (Section 2.4) — this keeps the parser itself from being the one closed-vocabulary component in an otherwise-generalizing pipeline.
2. **search.py** does image-level recall (top candidates by whole-image CLIP similarity), then garment-level recall for each parsed (category, color) — an **exact metadata filter** on the garment collection (expanded to the whole chromatic family) for deterministic recall, plus a fuzzy CLIP-embedding search for anything beyond exact/family parses. Final scoring is `score = W_CLIP·image_sim + W_COMP·attribute_overlap + W_SCENE·(scene+style+weather match)`, renormalized over whichever signals the query actually produced. `attribute_overlap` gives 1.0 for an exact color match, 0.8 for a same-family match, 0.6 for category-only, plus a bonus when a parsed layering relation is confirmed. Weights (0.35/0.55/0.10) were chosen by ablation over 120 real query combos, not guessed. All text a query needs embedded (the query itself, any zero-shot vocabulary candidates, and every garment sub-query) is batched into at most two model calls, not one call per piece — measured to be ~3x faster than the naive sequential approach on this CPU-only backbone (Section 2.5).
3. **If parsing finds nothing** (a genuinely novel description), the query degrades to pure whole-image CLIP similarity — the hybrid machinery only ever sharpens results, it never blocks a zero-shot query.

2.2 How this handles compositionality (the PRD's explicit hint)

Vanilla CLIP pools a whole image into one vector, so "red tie + white shirt" and "white tie + red shirt" — the same bag of visual concepts — embed almost identically. Three things fix this:

- **Per-garment crops are embedded and labeled independently.** A tie crop and a shirt crop in the same image are separate vectors with separate ground-truth colors; "tie=red" cannot bleed into "shirt=white" the way a single pooled embedding does.
- **Exact-match re-ranking is instance-scoped.** The attribute score only gives credit when a query's (category, color) pair matches a *specific garment instance*, not just "this image contains red and white somewhere."
- **Layering relations add a second axis of composition** ("a jacket over a shirt") on top of per-garment category/color.

The compositional win, measured. A controlled experiment runs the exact case the PRD hint names: for 60 corpus images containing two differently-colored garments, it builds the correctly-composed query and the color-swapped decoy ("red shirt + blue pants" vs. "blue shirt + red pants") and measures how often each system scores the correct phrasing higher:

System	Discrimination accuracy	Mean score margin
CLIP-only (whole-image similarity — the baseline the PRD says to beat)	0.650	+0.006

Hybrid (this system)	1.000	+0.400
----------------------	-------	--------

CLIP-only barely beats chance with a near-zero margin — precisely because a single pooled vector contains "red", "blue", "shirt", "pants" for *both* phrasings. The instance-scoped attribute re-ranking makes the distinction crisp. This is the single most on-point measurement for the assignment.

2.3 Context awareness: color + clothing type + location, measured directly

The PRD's Context Awareness requirement gives this exact example: "color + clothing type + location." An experiment builds the literal 3-attribute query for every (category, color, scene) triple with real support in the corpus (150 sampled), and compares it against the same query *without* the location term:

Query form	mean P@5	mean R@5
2-attribute (color + type only)	0.325 – 0.384	—
3-attribute (color + type + location)	0.624 – 0.689	0.887 – 0.969
Location term's effect on P@5	+0.299 to +0.305	reproduced across two independent runs

The location term isn't decorative — it roughly **doubles precision**, because color+category alone is often shared across many scenes (a black dress appears on runways, in studios, and on the street), and the scene score breaks that ambiguity.

2.4 Zero-shot capability — measured at three levels

- 1. Novel descriptions overall.** A qualitative probe uses 5 queries built from words absent from *every* vocabulary in the system (category list, color palette, scene/style/weather prompt banks) — the only thing that can answer them is fashion-CLIP's raw semantic generalization. "*An elegant evening gown for a gala*" correctly surfaces dress + formal + runway images despite "elegant" and "gala" appearing nowhere in any label. "*A cozy knit sweater for a winter morning*" correctly finds sweaters despite none of those words being seen. Abstract aesthetic judgments ("minimalist", "oversized") score visibly lower-confidence — honestly the hardest case, since there's no structured signal to lean on there, same as vanilla CLIP.
- 2. The query parser is itself zero-shot.** A structured parser is normally the one closed-vocabulary component that drags down an otherwise zero-shot system. `vocab_resolver.py` fixes this by resolving unknown garment words through fashion-CLIP itself. The hard part is precision, not recall — fashion-CLIP returns a high similarity for non-garment words too ("near", "person", "standing" score as high as real garments against category prototypes), so a plain threshold would inject false constraints and hurt retrieval. The fix is a **two-prototype zero-shot classifier**: embed the bare candidate word, and accept it as a garment only if its best similarity to the garment categories beats its best similarity to a bank of non-garment concept prototypes ("a place", "an action", "a person", "a time of day...") by a margin. Calibrated and locked into a regression test: **recall 8/8** (windbreaker→jacket, loafers→shoe, leggings→tights...), **precision 3/3** (filler sentences correctly resolve to no garment). Verified end-to-end: "a person in a navy windbreaker" parses to ('jacket', 'navy') and returns a navy jacket at rank 1.
- 3. Corpus-grounded benchmark.** 258 GARMENT-category (category, color) queries auto-generated from real corpus labels (not 5 hand-picked prompts) score **mean P@5 = 0.850, mean R@5 = 0.926** — evidence the system generalizes across the corpus's real attribute space, not just a few cherry-picked examples.

2.5 Honest evaluation: coverage, real bugs, latency, and shortcomings

Ground-truth coverage. Before trusting any Precision@k number, a coverage check verifies the ground truth even exists in the corpus. This corpus contains **zero yellow coats and zero red ties at all** — the PRD's evaluation queries

#1 and #5 ask for exact combinations that don't exist in this 3,200-image corpus. A retrieval system cannot return an image that isn't there, so Precision@5 on those two queries is capped below 1.0 regardless of retriever quality. (Qualitatively, the top result for the yellow-raincoat query is a genuinely bright yellow/gold raincoat — the best answer this corpus has.)

Precision@5 on the 5 PRD queries (coverage-aware): mean = **0.600**, with the two capped queries (#1, #5) explained above, and #3 (blue shirt on a park bench) landing at its exact ceiling of 0.20 because only one true "blue" shirt exists post-reclassification.

Query latency, measured with `eval/profile_latency.py`, went through three phases: an initial 15-55ms baseline; a 2-3x regression (75-190ms) after adding zero-shot vocabulary resolution and multi-color extraction, because each made small separate model calls; and a fix — batching every text a query needs embedded into at most two model calls total — that landed **below the original baseline (20-61ms)**, keeping all the added capability at no net latency cost. The batching effect itself was measured before implementing it: 4 sequential single-item embedding calls took ~~44ms~~ median on this CPU-only backbone, one batched 4-item call took ~14ms (3x faster).

Eval-iteration speed: every eval script used to recompute ground truth from raw images on every run (re-opening thousands of images, re-running color extraction) instead of reading what indexing already computed and stored. Measured directly: 385.8 seconds → 0.283 seconds (**1364x**) for the same data, by reading it back from the index instead. The whole eval suite now runs in well under a minute total.

Six real bugs/inefficiencies found and fixed during development, each caught by building an actual check rather than trusting first-pass output: (1) a khaki/army-green jacket was misclassified "yellow" by an early hue-only color rule, fixed by moving to Lab-space nearest-neighbor naming; (2) a genuine blue-shirt image ranked 142nd out of 7,800 in garment-crop similarity search, fixed with a deterministic exact-match recall pass; (3) the finer Lab-space color palette silently broke "blue" exact-match recall by splitting it into blue/navy/denim/sky-blue, fixed with chromatic-family partial credit; (4) the zero-shot vocabulary resolver was re-embedding words already classified elsewhere (e.g. "office"), measurably inflating latency, fixed by excluding already-classified token positions; (5) the remaining latency regression was root-caused to scattered sequential model calls and fixed by batching; (6) every eval script's redundant ground-truth recomputation, fixed as above.

Shortcomings, stated directly, not hidden: the attribute vocabulary is closed (46 categories, 35 colors) — mitigated by always falling back to CLIP similarity; color naming is nearest-neighbor over a fixed palette, not a trained model; spatial reasoning covers only "layered", not true z-order; corpus size still caps achievable precision on rare/absent combinations even after growing to 3,200 images; and the corpus's environment coverage is skewed toward runway/studio (Section 2.0), a property of the source dataset, not the retrieval algorithm.

2.6 Scalability to ~1M images

Both the image-level and garment-level searches pull a **fixed-size candidate pool**, not the full corpus, so query latency is architecturally flat as the dataset grows — measured directly (a latency profiler), not just asserted. The exact-filter recall pass is bounded by the number of matching instances for a given (category, color), still far below corpus size at any realistic scale. Every text embedding a query needs is batched into at most two model calls regardless of query complexity (Section 2.5), and the actual bottleneck at 1M images is indexing throughput (solved by batching on GPU — confirmed image embedding is compute-bound, not call-overhead-bound, so batch size alone can't fix it, GPU parallelism is the real lever) and Chroma's single-node persistence (solved by swapping in a sharded/managed vector DB — the retriever only calls the standard collection query API, so this is a client swap, not a rewrite).

2.7 On "state of the art"

There is no published benchmark for "natural-language query → Fashionpedia image retrieval with multi-attribute compositionality," so there is nothing concrete to claim victory against, and doing so would be dishonest. What this

submission reports instead: a controlled compositional experiment showing the hybrid system goes from CLIP-only's 0.65 (near chance) to 1.00 on the exact color-swap case the PRD names; a direct test of the PRD's own "color + type + location" example showing location roughly doubles precision; a measured, honestly-reported dataset composition audit against the PRD's own 3 required axes; a 258-query corpus-grounded benchmark; a 120-combo weight ablation; a zero-shot parser with calibrated precision/recall; six real bugs/inefficiencies caught by building actual checks, including a latency regression fixed to land below the original baseline and a 1364x eval-iteration speedup; a measured latency profile; and an explicit ground-truth coverage check so precision numbers mean what they claim to mean.

3. Codebase (GitHub) Link

[To be added: the author will push this repository to a GitHub repo and link it here before final submission.]

The codebase is complete and organized as:

- `common/` — shared configuration and the Lab-space color classifier (no logic, only constants both indexer and retriever depend on)
- `indexer/` — Part A: feature extraction and vector storage
- `retriever/` — Part B: query parsing and search logic
- `eval/` — coverage checks, benchmarks, ablations, and regression tests, all independently reproducible
- `docs/` — this write-up and the original PRD

Setup and reproduction instructions are in `README.md` at the repository root.

4. Approaches for Future Work

4a. Extending for locations (cities, places) and weather

Place-type and weather zero-shot tagging already ship as part of this submission (Section 2.1, item 5). City-level location was deliberately not attempted — nothing in a typical street-style photo honestly reveals which city it was taken in, and fabricating that structure would be dishonest rather than useful. Two concrete extensions:

- If real deployment metadata exists (EXIF, geolocation, capture timestamp), prefer that over any zero-shot inference — it would be strictly more reliable.
- Extend the prompt banks for finer place granularity (mall, restaurant patio, subway platform) — the tagging and scoring machinery already generalizes to any additional prompt-bank axis with no architecture change, just new prompts in a config file.

4b. Improving precision further

- **Swap the CLIP backbone.** `patrickjohncyh/fashion-clip` was chosen for being a well-established fashion-domain checkpoint; swapping to a larger one (e.g. Marqo-FashionSigLIP) is isolated to a single config change, since embedding is the only module that touches the model.
- **Grow the weight ablation.** The hybrid-scoring weights were chosen from a 120-combo ablation; as more labeled query data becomes available, a small learned re-ranker over the same three signals (CLIP similarity, attribute match, scene/style/weather match) could sharpen this past what a fixed ablation grid reaches.
- **Learn the color namer.** Color naming currently uses nearest-neighbor search over a fixed, hand-picked Lab-space reference palette. The infrastructure is the right foundation; the natural next step is training a namer on a labeled color dataset rather than hand-picking reference points.
- **Extend spatial relations past "layered."** True z-order ("tucked in" vs. "worn over") needs occlusion/depth signal beyond bounding-box overlap — e.g. instance segmentation overlap direction, or a small trained relation classifier.
- **Diversify the environment axis.** Section 2.0 shows office/home scenes are underrepresented in this Fashionpedia-derived corpus; supplementing with a street-style or lifestyle-photography dataset (rather than more runway/editorial images) would directly address that specific gap.
- **Grow the corpus further.** Corpus growth (1,158 → 3,200 images) is already implemented in this submission. The two lowest-scoring PRD evaluation queries are capped by combinations (a yellow coat, a red tie) that simply don't exist anywhere in this specific dataset — more or different source data is the remaining lever for those two cases, not the retrieval algorithm.